Credit Lecture 12: Foundation Models, In-Context Learning and the Cold Start

Deep Learning for Actuarial Modelling: a credit risk companion

AUTHOR

Mario Ervedosa, adapting Scognamiglio, Maggi, Wüthrich and Richman

PUBLISHED

September 4, 2026

ABSTRACT

Foundation models, tabular in-context learning and the in-context learning credibility transformer of Padayachy, Richman, Scognamiglio and Wüthrich (2025), asked one question throughout: does a prior the model never learned from this book, or context it retrieves at prediction time, buy what lecture 10-11's credibility gate did not? The yardstick is that lecture's seed standard deviation of 0.126. A pretrained prior pays where the sample is small and stops paying fast. TabPFN v2 leads a learning curve from 250 rows to 10,000, reaching 113.3 at 250 rows where the Credibility Transformer manages 119.6 against a null of 122.1, and where the maximum-likelihood GLM does not merely lose but separates, averaging 1,901. By 10,000 rows the three arms span 0.16 in total. Above all, TabPFN handed 10,000 rows matches a logistic GLM fitted on all 118,933 almost exactly, 110.88 against 110.87, so a cross-table prior substitutes for the lender's history without bettering the lender's model. It also arrives 1.9 percentage points below the observed default rate, about 2.3 sampling standard errors, and it has no parameter through which that can be corrected. The ICL-CT transfers with one part missing. Its decorator weights each retrieved outcome by exposure, and a fixed-horizon table has none, so the weight collapses to a constant and loses the borrower-level content that made it a credibility construction. Initialised at the fitted base model it reproduces that model exactly, which is the check to run before reading any attention weight, and from there five paired fits put in-context learning 0.012 behind the base model with a paired standard deviation of 0.032. In two of the five seeds no epoch beat the initialisation at all. Three measurements say why. The

paper’s batch retrieval unions 200 targets’ neighbourhoods and caps the union at 1,000, and since 95 per cent of retrieved rows are retrieved by exactly one target the cap hands the median borrower three of its own sixty-four neighbours. Scoring one borrower at a time against its own neighbours makes the attention row flatter still, a coefficient of variation of 0.02 against 0.10, without improving the deviance. And the neighbourhood mean that survives the averaging correlates 0.65 with what the base model already predicts. Credit’s version of the course’s limited-data problem is a cold start, and Bondora supplies a real one in a Slovak book of 296 loans, originated over ten months and then closed, defaulting at 70.6 per cent against Estonia’s 17.0. There every model under-predicts by forty percentage points, both networks are decisively worse than the null model at 236.6 and 234.6 against 195.8, and only the GLM beats it. Retrieval makes it worse rather than better, drawing 71.0 per cent of its context from Estonia against 58.1 per cent of the pool, because similarity is measured in the space of the very model that is wrong about the segment. That experiment also yields a methodological warning: the paired bootstrap reports a confident two-point gain under one training budget and a confident four-point loss under a longer one, with no resample dissenting either way, so an interval computed on 296 loans measures sampling variation and not the model-selection variation that dominates it. The governance bill is smaller than the construction suggests, and for the same reason the accuracy gain is. Re-drawing half the retrieval pool moves a borrower’s fitted log-odds by 0.002, against the 1.043 that separates lecture 9’s leading reason code from its second, so the explanation survives. An architecture attending sharply enough to buy accuracy would put that back at risk, which is why the re-draw is worth running rather than assuming.

Introduction

Every lecture in this series so far has fitted its model on the book it then scored. Lecture 12 breaks that pattern twice over. A **foundation model** carries a prior learned somewhere else entirely, and **in-context learning** supplies experience at prediction time rather than at fitting time, so neither one is fully described by the weights a lender would document.

The question this lecture asks is narrower than the course's, and it is set by what lectures 10 and 11 found. There the Credibility Transformer's explicit gate behaved exactly as described and bought no accuracy, its sweep spanning 0.213 against a seed standard deviation of 0.126, and its implicit credibility weight turned out to be a property of the fit rather than of the borrower. Consequently the honest question here is not whether in-context learning is impressive. It is whether retrieved context delivers what a learned prior did not. That standard deviation of 0.126 is the yardstick, and it is quoted again at every deviance table below.

Credit reframes the course's motivation as well. Insurance lecture 12 motivates credibility with small portfolios, new products, rare events, and new vehicle models. A lender's version is sharper, because lenders open and close whole books: a new country, a new product, a segment whose default history does not exist yet. Bondora supplies a real instance rather than a constructed one, and section 4 uses it.

▶ Imports and shared settings

1 Foundation models as a model class

1.1 What the term means

A **foundation model** is trained on a broad and diverse corpus and then transfers to many downstream tasks with minimal task-specific change. Three properties travel with the definition: scale, in parameters and in training tokens; general representations, meaning the learned features are not tied to one task; and versatile adaptation, meaning the same weights serve many jobs. In particular the third property is what makes the term useful, since a model that transferred only to tasks resembling its training corpus would be a pretrained model and nothing more.

The backbone is almost always a Transformer with autoregressive generation, which lectures 10 and 11 built. Capability then follows a joint power law in compute, data, and parameters (Kaplan et al., 2020). However that law was initially read as a licence to grow parameters, and Hoffmann et al. (2022) corrected it into the compute-optimal balance that told the field to budget tokens as well.

1.2 The model class

Write the input space as $\mathcal{X}$ and the output space as $\mathcal{Y}$. The model class is

$$\mathcal{F} = \{ f_\theta : \mathcal{X} \to \mathcal{Y} \mid \theta \in \Theta \}, \qquad \Theta \subset \mathbb{R}^r, \quad r \gg 10^8.$$

In the categorical case with K^{cls} classes the model output parametrises next-token probabilities through a softmax,

$$p_\theta(Y = k \mid \boldsymbol{x}) = \frac{\exp\{f_{k,\theta}(\boldsymbol{x})\}}{\sum_{l=1}^{K^{\text{cls}}} \exp\{f_{l,\theta}(\boldsymbol{x})\}}.$$

Pre-training maximises the autoregressive likelihood over a corpus of token sequences,

$$\theta^{\star}_{\text{pre}} \in \arg\max_{\theta \in \Theta} \sum_{t} \log p_{\theta}(Y = x_{t+1} \mid x_{1:t}),$$

which is the same maximum-likelihood principle lecture 2 applied to a logistic generalised linear model (GLM), differing in the size of Θ and in the fact that the response at step t is the covariate at step $t + 1$.

Note the notation. K^{cls} carries the class count here because section 3 needs K for the number of retrieved neighbours, and a twelve-month default response has $K^{\text{cls}} = 2$ throughout, so the softmax above reduces to the logistic link this series has used since lecture 2.

1.3 Adaptation, and what each rung means for a lender

Adaptation is where a foundation model becomes a credit model, and the options form a ladder from cheapest to most invasive. Note that the rungs differ in what they leave a lender owning, which matters more here than their compute cost.

- **In-context learning (ICL).** Put demonstrations $\mathcal{C}$ in the input and evaluate $f_{\theta^{\star}_{\text{pre}}}([\mathcal{C}, x])$ (Brown et al., 2020). No weight changes at all.
- **Retrieval-augmented generation (RAG).** Fetch relevant documents and condition on them. Again no weight changes.
- **Parameter-efficient fine-tuning (PEFT), including low-rank adaptation (LoRA).** Train a small number of added parameters and freeze the rest.
- **Full fine-tuning.** Train everything, which is the most expensive rung and the only one that produces a model whose weights are wholly yours.

Each rung reads differently in a lending context, and the first thing worth saying is that lenders have used the bottom rung for decades without calling it that. A **bureau score** is a pretrained prior, since somebody else fitted it, on a population you cannot inspect, and you consume it as a covariate. Consequently the governance apparatus around bureau scores, meaning the vendor's population documentation and the lender's own monitoring, is the closest existing precedent for what section 5 argues a tabular foundation model would need.

Similarly RAG has an obvious home over a credit policy manual, where the question "does this application breach policy?" is a retrieval problem before it is a modelling one. That use is a document task rather than a scoring task, so it sits outside this series' scope. Nevertheless it is named here, because it is the rung most likely to reach a lending operation first.

In-context learning over a retrieved set of similar borrowers is the rung sections 3 and 4 build, and it is the one that changes what a credit decision is made of.

1.4 The GPT series, briefly

The trajectory matters mainly as evidence that adaptation improves with scale. GPT-1 and GPT-2 established that unsupervised pre-training transfers (Radford et al., 2018, 2019). GPT-3 showed that few-shot prompting alone covers a broad task range without gradient updates (Brown et al., 2020), which is the result that made in-context learning a research programme rather than a curiosity. InstructGPT added alignment through reinforcement learning from human feedback (Ouyang et al., 2022), and GPT-4 broadened reasoning and robustness (OpenAI, 2023).

Two sensitivities travel with the whole family and both matter to anyone tempted to put a language model near a credit decision. Prompt format and demonstration order change the answer, so the same question asked twice can be answered differently. Furthermore the mechanism is next-token prediction, so an arithmetic or regulatory claim arrives as a plausible continuation rather than as a derivation. Neither observation argues against the technology; both argue against treating its output as a calculation.

2 Tabular foundation models

2.1 Why tabular data resists the recipe

Language and images gave foundation models two gifts that a credit application table withholds. Tokens share a vocabulary across every document, and pixels share a grid across every image, so a model pretrained on one corpus meets familiar inputs in the next. A table's columns share nothing: `IncomeTotal` in euros and `Education` as a code are different kinds of object, they arrive in an arbitrary order, and the next lender's table has different columns entirely.

Four consequences follow. Features are heterogeneous and mixed in type. Furthermore missingness is informative rather than incidental, as lecture 2's `HomeOwnershipType` showed. There is no natural order across columns, so a model must be permutation invariant or else learn an ordering that means nothing. Above all, gradient-boosted trees set a high baseline that deep tabular models have repeatedly failed to clear, whereas language and vision offered no comparable incumbent.

The foundation approach answers by pre-training a **cross-table prior**: a model learns, over many synthetic tables, what tabular relationships tend to look like, and then reuses that prior on a table it has never seen.

2.2 The families

Architecture	Idea	Reference
TabTransformer	Tokenise categorical features, attend over them	Huang et al. (2020)
FT-Transformer	Tokenise continuous features too	Gorishniy et al. (2021)
TransTab	Cross-table transfer with aligned embeddings	Wang and Sun (2022)
TabPFN	Prior-data fitted network, ICL in one forward pass	Hollmann et al. (2022)
TabPFN v2	Tabular foundation model, wins to $n \approx 10^4$	Hollmann et al. (2025)
TabICL	Two-stage scaling to $n \approx 5 \times 10^5$	Qu et al. (2025)

One attribution is worth correcting, because the course misplaces it and this series has already checked the primary text. Feature tokenisation, meaning the treatment of a continuous covariate as a token through a small network, belongs to the **FT-Transformer** of Gorishniy, Rubachev, Khrulkov and Babenko (2021). Lecture 10-11 section 3.2 uses that construction and cites it correctly, and `notes/transformer-lecture-structure.md` records the verification. In contrast the course attributes it to TabM, which proposes no tokeniser at all, so the error is one of attribution rather than of substance.

2.3 TabPFN as approximate Bayesian inference

The prior-data fitted network is the family's clearest idea, and it is easiest to state as a Bayesian one. Given a context dataset $\mathcal{D}$ and a query $\boldsymbol{x}^\star$, the posterior predictive is

$$p(y^\star \mid \boldsymbol{x}^\star, \mathcal{D}) = \int p(y^\star \mid \boldsymbol{x}^\star, \boldsymbol{\theta})\, p(\boldsymbol{\theta} \mid \mathcal{D})\, \mathrm{d}\boldsymbol{\theta},$$

which is intractable for any interesting model class. TabPFN instead learns a function $f_\psi(\boldsymbol{x}^\star, \mathcal{D})$ to approximate that posterior directly, by minimising expected negative log-likelihood over tasks drawn from a synthetic prior (Müller et al., 2021). The training distribution is therefore not data at all. It is a prior over data-generating mechanisms, and the network learns to do inference under it.

At prediction time the whole context dataset is fed in alongside the query, and one forward pass returns the predictive distribution. Consequently fitting, in the usual sense, does not happen. TabPFN v1 alternates attention over columns and over rows, which handles the missing column order and yet costs quadratically in the number of rows, and that cost is what bounds the family's sample size. Alternatively TabICL embeds each row to a fixed dimension first and then runs in-context learning over those compact embeddings, which reaches about 5×10^5 rows (Qu et al., 2025).

> **WHAT "NO FITTING" DOES AND DOES NOT MEAN**
>
> TabPFN performs no gradient descent on your data, and yet the context dataset determines the prediction completely. So the estimation has moved rather than vanished: it happens inside the forward pass, using weights fitted on synthetic tasks. For a lender the practical consequence is that there is no fitted parameter vector to document, monitor or re-estimate, which section 5 takes up.

2.4 The Bondora book

The frame, the filters, and the seed-1 split below are those of lectures 4 to 11, so the deviances chain to theirs without rescaling.

▶ The Bondora modelling frame, unchanged from lectures 4 to 11

```
n = 148,667,  learn 118,933 / test 29,734,  learning-sample default rate
0.28878
```

2.5 The learning curve

The course claims these architectures are strong where the sample is small and may struggle on unbalanced data. Both halves are testable here, so this section tests them.

The design is a nested learning curve. Three independent subsamples of the learning sample are drawn at each of six sizes, and at each size three models are fitted: lecture 2's logistic GLM, the Credibility Transformer of section 3, and TabPFN v2. Every model is then scored on one fixed test sample so the arms are comparable.

One departure from the rest of the series is forced and is worth stating plainly. TabPFN's cost grows quadratically in the size of its context, and scoring all 29,734 test rows at every point on the curve would dominate the render, so the curve is scored on a fixed random 3,000-row subsample of that test set. Consequently the curve's deviances are comparable **with each other** and sit slightly above the series' full-test figures. Section 3 returns to the full test set for the numbers that chain.

▶ The three arms of the learning curve

> **TWO TRAPS IN THE TABPFN PACKAGE**
>
> The installed `tabpfn` resolves by default to its newest checkpoint, whose weights sit behind a gated licence on HuggingFace and fail without an accepted licence and a token. Naming `tabpfn-v2-classifier.ckpt` selects the **v2** weights, which download without credentials and are the version the course cites (Hollmann et al., 2025).
>
> Pinning the `tabpfn` 2.x release instead is the obvious alternative and it breaks the environment: it downgrades `scikit-learn` to 1.6.1, which is not the course's pin, and restoring 1.8.0 then breaks `tabpfn` outright. The package is installed on top of the course requirements and changes none of them; `notes/foundation-models-lecture-structure.md` records the detail.

The third arm is lectures 10 and 11's Credibility Transformer, imported unchanged. That lecture owns the architecture, so the cell below is folded and cited rather than re-derived; section 3 explains only the parts this lecture adds.

▶ Lecture 10-11's Credibility Transformer, unchanged

▶ Running the curve: three sizes-by-draws grid, three arms

```
mean out-of-sample deviance over three draws, 3,000-row test subsample
            GLM       CT   TabPFN     null
n
250    1901.015  119.638  113.312  122.114
500    2017.065  117.694  112.100  122.114
1000   1370.436  116.481  111.973  122.114
2000   1299.679  112.951  111.575  122.114
5000    111.568  112.170  111.367  122.114
10000   111.150  111.093  110.988  122.114

mean AUC over three draws
       GLM_auc  CT_auc  TabPFN_auc
n
250     0.5685  0.5949      0.6857
500     0.5720  0.6547      0.6957
1000    0.5860  0.6727      0.6997
2000    0.6143  0.6877      0.7031
5000    0.7000  0.6937      0.7071
10000   0.7020  0.7032      0.7069
```

Three readings follow, and the first is the one the course predicts.

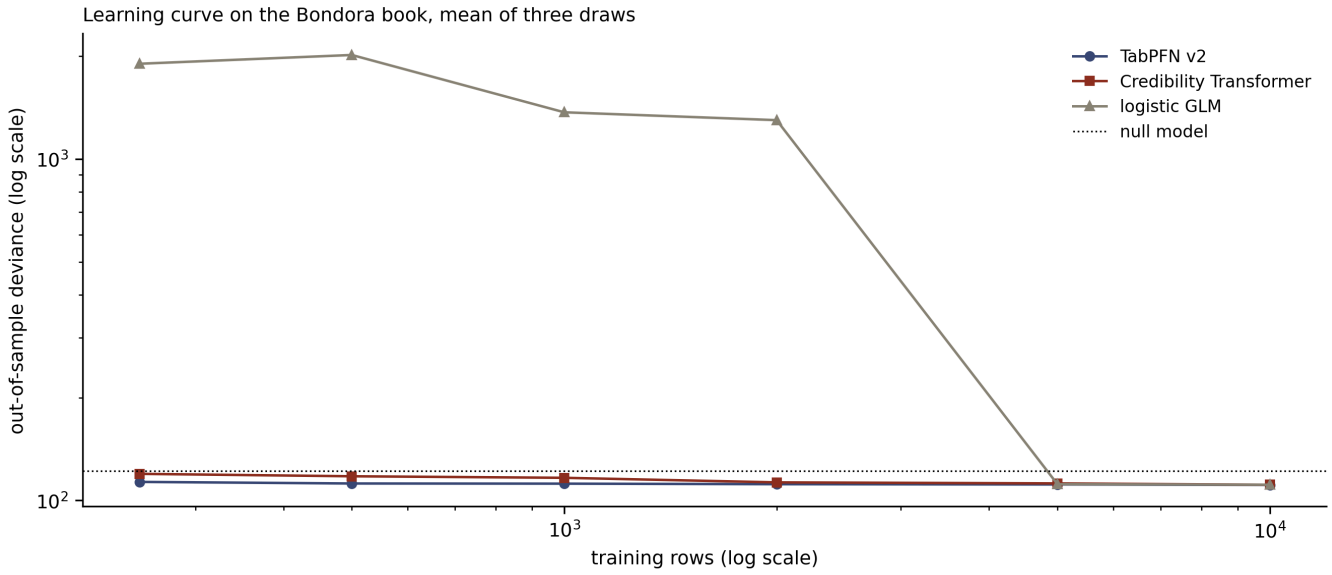
TabPFN leads wherever the sample is small, and its margin closes fast. At 250 training rows it reaches 113.3 against the Credibility Transformer's 119.6, on a subsample whose null model scores 122.1. So the network closes 2.5 of the 8.8 deviance points the foundation model closes against that null, a little over a quarter, on identical data. By 10,000 rows the three arms span 0.16 of a deviance point in total, which is the same order as the seed spread of 0.126 and therefore not a ranking worth reading. Consequently the pretrained prior behaves exactly as advertised: it substitutes for data a lender does not have, and it stops paying once the lender has enough. Similarly the area under the receiver operating characteristic curve (AUC) tells the story more starkly, since TabPFN reaches 0.686 on 250 rows where the network manages 0.595 and the GLM 0.569, and all three converge to about 0.703 by 10,000.

The GLM does not merely lose at small n; it separates. Its mean deviance runs from 1,901 at 250 rows to 1,300 at 2,000 before collapsing to 111.6 at 5,000, because maximum likelihood on around thirty parameters and a few hundred observations drives

coefficients towards infinity whenever a covariate pattern is perfectly classified. A fitted probability of essentially zero attached to an observed default then costs the deviance without limit. The behaviour is erratic as well as bad, since whether any pattern separates depends on the draw. This is worth dwelling on, because it is the practical case for a prior rather than a rhetorical one: the classical estimator has no mechanism to shrink, so on a genuinely small book it fails loudly and needs either penalisation, classing (lecture `F1`), or a prior from somewhere else.

The plot below shows the shape. Note the logarithmic vertical axis, without which the GLM's small-sample behaviour would flatten everything else into a line.

▶ The learning curve



2.6 What twelve times the data is worth

The curve stops at 10,000 rows because TabPFN's cost grows quadratically in its context and its pretraining regime does not extend much beyond that. The comparison a lender actually faces is therefore between TabPFN at its ceiling and a network trained on everything.

▶ TabPFN at its ceiling against the full-sample Credibility Transformer

```
on the 3,000-row test subsample
  TabPFN v2, 10,000 context rows   dev  110.877   AUC 0.7085
  GLM,       118,933 rows          dev  110.873   AUC 0.7031
  CT,        118,933 rows          dev  110.363   AUC 0.7107

on the full 29,734-row test sample, which is the series' scale
  CT,        118,933 rows          dev  107.342   AUC 0.7250   early stop
149
```

Two comparisons come out of that cell and the second is the more useful one.

TabPFN with 10,000 context rows reaches 110.88, against 110.36 for a Credibility Transformer trained on all 118,933. Twelve times the data is therefore worth about half a deviance point here, which is four times the seed standard deviation of 0.126 and so a real difference, and it is also small enough to be worth stating out loud.

More striking, TabPFN with 10,000 context rows matches the logistic GLM fitted on all 118,933 rows almost exactly, 110.88 against 110.87. Hence on this book a foundation model handed 8 per cent of the data, and given no fitting at all, reproduces the classical model a lender would build on all of it. That is the clearest statement of what a cross-table prior is worth on credit application data: it is a substitute for the modeller's time and for the lender's history, and it is not a substitute for a better model.

2.7 Calibration, and the balance property

Discrimination is only half of what a probability of default has to deliver. Lecture 7 established the **balance property**, namely that a model's fitted probabilities should sum to the observed number of defaults, and showed why a portfolio-level bias survives any amount of ranking accuracy. TabPFN carries no offset, no exposure, and no intercept a modeller can set, so there is no mechanism inside it that enforces the property.

One caution has to come before the table. The comparison sample holds 3,000 loans, so its own observed default rate carries a standard error of about 0.8 percentage points, and the learning sample's rate differs from it by about one percentage point through

sampling alone. A shortfall is therefore only worth calling a bias if it is large against that standard error.

▶ Portfolio-level calibration on the test subsample

```
sampling standard error on the observed rate: 0.00836
                         model  mean predicted PD  gap to observed  gap /
standard error
 observed, 3,000-row subsample            0.29933              NaN
NaN
     observed, learning sample            0.28878              NaN
NaN
TabPFN v2, 10,000 context rows            0.28024         -0.01909
-2.28299
              CT, 118,933 rows            0.29116         -0.00818
-0.97788
             GLM, 118,933 rows            0.28972         -0.00962
-1.15004

on the full 29,734-row test sample
  observed          0.29209
  CT, 118,933 rows  0.28924   gap -0.00285
```

TabPFN lands 1.9 percentage points below the observed rate, which is 2.3 sampling standard errors and so a real shortfall rather than noise, though a modest one. The Credibility Transformer's gap is about two-fifths of that at 1.0 standard error and the GLM's is similar at 1.2, so neither is distinguishable from calibrated on this sample. On the full test sample, where the standard error is a quarter as large, the Credibility Transformer sits 0.3 percentage points below the observed rate.

Consequently a TabPFN score is usable for ranking and wants lecture 7's recalibration before any exposure is priced off it. That is the same treatment lecture 7 applied to the plain network, for the same reason, with one difference that matters: the network could be given an offset or refitted with a balance correction, whereas TabPFN has no parameter to correct, so the recalibration has to sit outside the model as a separate mapping the lender owns and monitors.

3 The in-context learning credibility transformer

Section 2's foundation model carries a prior learned on synthetic tasks. This section takes the opposite route to the same idea: keep a model fitted on this book, and give it, at prediction time, the observed outcomes of the most similar borrowers. That is the **in-context learning credibility transformer** (ICL-CT) of Padayachy, Richman, Scognamiglio and Wüthrich (2025). Consequently it is built in five components on top of lecture 10-11's model, and each is taken in turn below.

3.1 Component 0: the base credibility transformer

The base model is lecture 10-11's Credibility Transformer at $\alpha = 0.95$, fitted in this lecture rather than quoted from that one, because a single fit on this book is one draw from a distribution whose standard deviation is 0.126 and a copied number would not reproduce. It does two jobs. Its classification (CLS) token embedding, which lecture 10-11 built as the single token the decoder reads, is the space in which similar borrowers are found. Furthermore its decoder is frozen and reused, so the ICL mechanism is confined to that CLS token space.

It is the same fit section 2.7 already ran, reused here rather than repeated, since a second identical fit would cost two minutes of render time and produce the same weights.

```
base_ct, base_stop = m_full, stop_full
p_base_l = predict(base_ct, D, idx_learn)
p_base_t = p_full_all
print(f"base CT, alpha = 0.95, seed 100, early stop {base_stop}")
print(f"  in-sample {dev(p_base_l, yl):.3f}   out-of-sample "
      f"{dev(p_base_t, yt):.3f}   AUC {roc_auc_score(yt, p_base_t):.4f}")
```

```
base CT, alpha = 0.95, seed 100, early stop 149
  in-sample 105.502   out-of-sample 107.342   AUC 0.7250
```

3.2 Component 1: context retrieval

Similarity is measured in the base model's CLS token space by cosine similarity, which is the paper's choice and a sensible one here: the CLS token is what the decoder reads, so two borrowers with nearby CLS tokens are borrowers the fitted model treats alike.

For each target the top $K = 64$ neighbours are taken, the results are unioned across a target batch of $m = 200$, and the union is capped at $c = 1000$ context instances by keeping those retrieved most often.

```
def cls_tokens(model, D_, idx, batch=20000):
    """The CLS token for each row, which is the retrieval space."""
    out = []
    with torch.no_grad():
        for s in range(0, len(idx), batch):
            chunk = idx[s:s + batch]
            c_trans, _, _ = model.tokens(torch.tensor(D_["Xc"][chunk]),
                                         torch.tensor(D_["Xn"][chunk]))
            out.append(c_trans)
    return torch.cat(out)


def retrieve(Z_target, Z_pool, K=64, c_max=1000, exclude=None):
    """Top-K cosine neighbours per target, unioned over the batch, capped
        at c_max."""
    A_ = torch.nn.functional.normalize(Z_target, dim=1)
    B_ = torch.nn.functional.normalize(Z_pool, dim=1)
    S = A_ @ B_.T
    if exclude is not None:
        S = S.masked_fill(exclude.unsqueeze(0), -np.inf)
    top = torch.topk(S, k=min(K, S.shape[1]), dim=1).indices.reshape(-1)
    uniq, counts = torch.unique(top, return_counts=True)
    if len(uniq) > c_max:
        uniq = uniq[torch.topk(counts, k=c_max).indices]
    return uniq
```

THE RETRIEVAL LEAK

During training the targets sit inside the pool the context is drawn from, and a borrower is always its own nearest neighbour. Retrieving a target as its own context would therefore hand the model that target's observed response, and the result would score beautifully in the backtest while predicting nothing. Hence the `exclude` mask above. Indeed this is the same class of error as lecture 10-11's mis-sorted panel, because it is a leak that validates cleanly while the validation shares it.

3.3 Component 2: the outcome token decorator, and the exposure that is not there

The decorator is the mechanism's credibility half. Each context instance's CLS token is nudged by a learnable embedding of its observed response,

$$\boldsymbol{c}^{\text{decor}}(\boldsymbol{x}_j) = \hat{\boldsymbol{c}}^{\,\text{cred}}(\boldsymbol{x}_j) + \frac{v_j}{v_j + \kappa}\, \boldsymbol{z}^{\text{FNN1}}(Y_j),$$

where v_j is exposure and $\kappa > 0$ is the credibility coefficient. Targets are never decorated, since their responses are what the model has to predict. Exposure enters only through the weight $v/(v+\kappa)$ rather than as a covariate, which is what stops the construction leaking the response scale.

That weight is what makes this a credibility construction. A policy observed for three years contributes its outcome nearly in full; one observed for a month contributes a shrunken version. In an insurance pricing table, where v is exposure in policy-years and varies genuinely across rows, the weight carries real per-policy content.

Bondora's fixed-horizon table has no exposure. Lecture `D1` owns this point: `default_12m` is an indicator over a twelve-month window that every seasoned loan carries identically by construction, so there is no v_j to put in the formula. Setting $v \equiv 1$ collapses the weight to the constant $1/(1+\kappa)$, and κ stops being a credibility coefficient and becomes a plain shrinkage scalar applied to every retrieved outcome alike.

Two alternatives exist and both belong to other lectures. The survival table's observed duration is the honest exposure analogue, and it changes the estimand, which is the `S` track's subject. The loan `Amount` would make the decorator a loss-weighted construction rather than a credibility one, which is exposure at default rather than statistical exposure. Neither is substituted in here, because manufacturing a v would make the formula look transferable when the transfer is precisely what fails.

```
class ICLCT(torch.nn.Module):
    """Base CT plus the outcome decorator and two causally masked ICL
        layers."""

    def __init__(self, base, b=8, kappa=1.0, n_layers=2, seed=100):
        super().__init__()
        torch.manual_seed(seed + 7)
        self.base, self.b, self.kappa = base, b, kappa
        self.resp = torch.nn.Embedding(2, b)      # z^FNN1, the response
        embedding
        self.layers = torch.nn.ModuleList([ICLLayer(b) for _ in
        range(n_layers)])

    def decorate(self, c_ctx, y_ctx, v_ctx=None):
        """Component 2. With v = 1 the weight is the constant 1 / (1 +
        kappa)."""
        v = torch.ones(len(y_ctx)) if v_ctx is None else v_ctx
        w = (v / (v + self.kappa)).unsqueeze(1)
        return c_ctx + w * self.resp(y_ctx.long())

    def forward(self, c_tgt, c_ctx, y_ctx, v_ctx=None, want_A=False):
        ctx = self.decorate(c_ctx, y_ctx, v_ctx)
        h, A = c_tgt, None
        for layer in self.layers:
            h, A = layer(h, ctx)
        return (self.base.decode(h), A) if want_A else
        self.base.decode(h)
```

3.4 Component 3: causal self-attention

The context and target tokens are concatenated and a mask blocks target-to-target attention, so a target reads the context and itself and nothing else. Without the mask the

targets in a batch would read each other's tokens and, through their shared neighbours, their own responses.

```
class ICLLayer(torch.nn.Module):
    """One causally masked attention layer: targets query [context |
        self]."""

    def __init__(self, b):
        super().__init__()
        self.norm_in = torch.nn.LayerNorm(b)
        self.WQ, self.WK, self.WV = (torch.nn.Linear(b, b) for _ in
        range(3))
        self.norm_ff = torch.nn.LayerNorm(b)
        self.ff = torch.nn.Linear(b, b)
        # Identity initialisation: see section 3.6.
        for lin in (self.WV, self.ff):
            torch.nn.init.zeros_(lin.weight)
            torch.nn.init.zeros_(lin.bias)

    def forward(self, c_tgt, c_ctx):
        m = c_tgt.shape[0]
        Qn, Kn = self.norm_in(c_tgt), self.norm_in(c_ctx)
        Q = torch.tanh(self.WQ(Qn))
        K = torch.cat([torch.tanh(self.WK(Kn)), torch.tanh(self.WK(Qn))])
        V = torch.cat([torch.tanh(self.WV(Kn)), torch.tanh(self.WV(Qn))])
        A0 = Q @ K.T / Q.shape[-1] ** 0.5
        mask = torch.zeros_like(A0)
        mask[:, c_ctx.shape[0]:] = torch.where(~torch.eye(m,
        dtype=torch.bool),
                                               -np.inf, 0.0)
        A = torch.softmax(A0 + mask, dim=-1)
        skip = c_tgt + A @ V
        return skip + self.ff(self.norm_ff(skip)), A
```

3.5 Component 4: the frozen decoder

The decoder mapping the CLS token to a probability is taken from the base model and held fixed. The paper's argument for freezing it is an information bottleneck, because the ICL mechanism must then express itself as a movement within the CLS token space that the existing decoder already reads. Hence the base model's calibration is preserved

and what in-context learning is allowed to do is regularised, and section 3.6's initialisation makes that concrete.

3.6 The proposition, and its arithmetic

For target i the causal attention head produces

$$\boldsymbol{h}_i = a_{i,i}\, \boldsymbol{z}_V^{\mathrm{FNN}}\big(\hat{\boldsymbol{c}}^{\,\mathrm{cred}}(\boldsymbol{x}_i)\big) + \sum_{j \in \mathcal{J}_{\mathrm{context}}} a_{i,j}\, \boldsymbol{z}_V^{\mathrm{FNN}}\big(\boldsymbol{c}^{\mathrm{decor}}(\boldsymbol{x}_j)\big)\,,$$

with $a_{i,j} \geq 0$, with $a_{i,i} + \sum_j a_{i,j} = 1$, and with $a_{i,j} = 0$ for every j that is another target. So the head returns a credibility blend of the target's own signal and the outcome-enriched signals of its neighbours, and the blend weights are the attention row.

Indeed this is exact algebra rather than analogy, exactly as lecture 10-11 found for the CLS token's own row. Nevertheless it is worth checking numerically before any weight is interpreted, because an arithmetic identity that fails in code is a masking bug rather than a proposition. Two things must hold: the row must sum to one, and the target-to-target weights must be exactly zero.

3.7 Identity initialisation

Zeroing the value and feed-forward weights of each ICL layer makes the layer return its target token untouched, so before any ICL training the model reproduces the base model's fitted probabilities exactly. Lecture 9 initialises its LocalGLMnet at the fitted GLM for the same reason: it turns a comparison between two models into a measurement of what one mechanism adds. The gradients survive, since the derivative of the loss with respect to $\boldsymbol{W}_V$ is not zero at $\boldsymbol{W}_V = 0$.

The consequence for reading section 3.9 matters. Early stopping selects the best validation state including the initialisation itself, so an ICL-CT that matches the base model to three decimals is reporting that no epoch of in-context learning beat the state it started from.

3.8 The three training phases

Phase one is the base model above. Phase two inserts the decorator and the ICL layers, freezes the base model outright and trains the new parts alone, which keeps the retrieval space fixed and lets the CLS tokens be computed once. Phase three unfreezes the encoder, keeps the decoder frozen and fine-tunes everything at a tenth of the learning rate.

One departure from the paper is deliberate. Freezing the whole base model in phase two, rather than the decoder alone, is what makes the retrieval space constant during that phase; the paper's retrieval is computed in the base model's space in any case, so this removes a moving part rather than adding one. Validation is scored on a **fixed** set of context draws, re-seeded on every call, because re-drawing the context each epoch makes the early-stopping signal noisy enough to select an epoch at random.

▶ Phases two and three

3.9 Fitting it, and checking the proposition

```
Z_learn = cls_tokens(base_ct, D, idx_learn)
Z_test = cls_tokens(base_ct, D, idx_test)
y_learn_t = torch.tensor(yl, dtype=torch.float32)

fresh = ICLCT(base_ct, b=base_ct.b, kappa=1.0, seed=100)
p_init = predict_icl(fresh, Z_learn, y_learn_t, Z_test)
print(f"identity check: max |p_ICL - p_base| = {np.abs(p_init -
        p_base_t).max():.2e}"
      f",  deviance {dev(p_init, yt):.3f} against the base model's "
      f"{dev(p_base_t, yt):.3f}")

icl_ct, hist = train_icl(base_ct, D, Z_learn, seed=100, kappa=1.0)
Z_learn2 = cls_tokens(icl_ct.base, D, idx_learn)
Z_test2 = cls_tokens(icl_ct.base, D, idx_test)
p_icl_t, own_w, ctx_rows = predict_icl(icl_ct, Z_learn2, y_learn_t,
        Z_test2,
                                       want_own=True, pool_idx=idx_learn)
print(f"\nICL-CT out-of-sample {dev(p_icl_t, yt):.3f}   "
      f"AUC {roc_auc_score(yt, p_icl_t):.4f}")
for ph in ("2", "3"):
    h = hist[ph]
    print(f"  phase {ph}: baseline {h['baseline']:.4f}, best
        {h['val']:.4f} "
          f"at epoch {h['epochs']}")
    print(f"           trace {h['trace']}")
```

```
identity check: max |p_ICL - p_base| = 1.19e-07,  deviance 107.342
against the base model's 107.342

ICL-CT out-of-sample 107.342   AUC 0.7250
  phase 2: baseline 107.2837, best 107.2837 at epoch 0
           trace [107.3751, 107.4699, 107.3549, 107.326, 107.3175,
107.4017, 107.3785, 107.3925]
  phase 3: baseline 107.2837, best 107.2837 at epoch 0
           trace [107.3104, 107.3394, 107.3026, 107.2996, 107.3185]
```

▶ The attention row: does it sum to one, and is it flat?

```
context instances retrieved for this batch: 1000
  max |row sum - 1|            3.58e-07
  max target-to-target weight  0.00e+00
  a_ii mean                    9.23e-04  (sd 1.60e-04)
  context weight mean          9.99e-04
  context weight sd            9.96e-05  (coefficient of variation 0.100)
  largest context weight       1.86e-03  = 1.86 x the mean
  a uniform row would give     9.99e-04 to every entry
```

The proposition's arithmetic holds. Row sums sit within a few parts in ten million of one, and the target-to-target weights are exactly zero, which says the mask does what section 3.3 claims.

The weights themselves say something the proposition does not. The row spreads over about a thousand retrieved neighbours and one self token, so the target's own signal receives roughly a thousandth of the attention row. Consequently the "credibility blend between the target's own signal and the context" puts essentially all of its weight on other borrowers, and the applicant's own characteristics reach the decoder through the residual connection instead. Furthermore the context weights are close to uniform: their coefficient of variation is about a tenth, and the largest is under twice the mean. Retrieval has therefore selected a neighbourhood, and inside that neighbourhood attention is barely distinguishing between its members.

3.10 Results against the series

Five seeds, each fitting a base model and then an ICL-CT on top of it. The comparison is paired within seed, which is what makes a difference of a tenth of a deviance point readable at all against a seed standard deviation of 0.126.

▶ Five paired fits

```
 seed      base       icl  ph2  ph3      a_ii  ICL minus base
  100  107.3420  107.3420    0    0    0.0009         −0.0000
  101  107.2879  107.3266    5    1    0.0011          0.0387
  102  107.3298  107.3129   12    0    0.0010         −0.0169
  103  107.2647  107.2487    0    2    0.0010         −0.0160
  104  107.4993  107.5512    6    0    0.0011          0.0519

base CT over five seeds     mean 107.345   sd 0.092
ICL−CT over five seeds      mean 107.356   sd 0.115
paired difference           mean +0.0115   sd 0.0319
```

The answer is a clean null, and the way it is null is worth reading off the table rather than off the means alone.

Over five seeds the base model averages 107.345 and the ICL-CT averages 107.356, so in-context learning moves the paired difference by $+0.012$ with a standard deviation of 0.032 across seeds. Pairing matters here: the between-seed spread of either model is about 0.09, and lecture 10-11 measured 0.126 over ten seeds, so an unpaired comparison could not resolve a tenth of a point. The paired comparison can, and what it resolves is a difference of about a third of its own standard error in the wrong direction.

The best-epoch columns say why, and they say it in two different ways depending on the seed. At seeds 100 and 103 no epoch of phase two improved on the identity initialisation at all, so the model selection returned the base model unchanged and the deviances match to three decimals by construction. At seeds 101, 102 and 104 phase two did find a better validation state, and that improvement failed to reach the test sample: seed 101 gained on validation and lost 0.039 out of sample, and seed 104 lost 0.052. Consequently the mechanism either declines to train or trains into noise, and on this book neither outcome produces an improvement a lender could bank.

WHAT A NULL RESULT HERE DOES AND DOES NOT LICENSE

This is a statement about Bondora application data with eight covariates, at $n = 118{,}933$, against a base model that was already strong. It is not a statement about the ICL-CT on French motor insurance, where Padayachy et al. (2025) report a gain, and section 3.11 gives a mechanical reason why the two books might genuinely differ. What travels is the method: initialise at the base model, pair by seed, and report the best epoch, because without those three a difference of a tenth of a point is unreadable either way.

3.11 Why the mechanism finds nothing to add

Three explanations are available, they are separable by measurement, and this section separates them. All three need a model that is actually doing in-context learning, so the first cell fits one: phase two to a fixed epoch budget, keeping the final state rather than the state section 3.8's selection rule chose. Section 5.2 reuses it.

▶ An actively trained ICL state

```
actively trained state: out-of-sample 107.362 against the base model's
107.342
```

3.11.1 The batch context is barely the borrower's own

Section 3.1's retrieval takes each target's top $K = 64$ neighbours, unions them across a batch of $m = 200$ targets, and caps the union at $c = 1000$ by keeping the rows retrieved most often. That is the paper's procedure. Whether it delivers a borrower its own neighbours depends on how much the 200 neighbourhoods overlap, which is a property of the book rather than of the architecture, so it is worth measuring.

▶ How much of a borrower's own neighbourhood survives the cap

```
200 targets x 64 neighbours = 12,800 draws
  distinct rows in the union            12,145
  context cap                           1,000
  retrieval frequency: mean 1.05, max 2, share retrieved exactly once
94.6%
  share of a target's own top-64 surviving the cap: mean 0.129, median
0.047, range 0.000 to 0.766
```

The 200 neighbourhoods barely overlap on this book. Almost 95 per cent of retrieved rows are retrieved by exactly one target and none by more than two, so the 12,800 draws collapse to about 12,100 distinct rows and the cap keeps roughly 8 per cent of them on a frequency criterion that is close to a tie-break. Consequently the median borrower is handed **three of its own sixty-four nearest neighbours**, and the mean is about eight.

So the context a borrower actually receives is close to a random thousand-row sample of the learning book. That is not a bug in this implementation, since the batching is the paper's; it is a scale mismatch, because a union of 200 neighbourhoods only fits inside 1,000 slots when neighbourhoods overlap heavily, and on 118,933 loans described by eight covariates they do not.

3.11.2 Per-borrower context does not sharpen the row

The obvious next question is whether the flatness of section 3.9's attention row is caused by that dilution. Scoring one target at a time makes the context exactly its own 64 neighbours, which is the sharpest context the mechanism can be given.

▶ Batch context against per-borrower context

```
batch context, 200 targets: 1000 context instances
  a_ii mean 9.06e-04   uniform would be 9.99e-04
  context weight coefficient of variation 0.099   largest / mean 1.97
  deviance on 400 test rows 102.024   base 102.052   largest move 0.0195
per-borrower context, one target: 64 context instances
  a_ii mean 1.50e-02   uniform would be 1.54e-02
  context weight coefficient of variation 0.022   largest / mean 1.02
  deviance on 400 test rows 102.201   base 102.052   largest move 0.0419
```

It does not. Given its own 64 neighbours the row is **flatter**, with a coefficient of variation of about 0.02 against 0.10 for the batch context, and the largest weight within 2 per cent of the mean. The prediction does move further from the base model's, roughly twice as far, and it does not move to a better place: the deviance on those 400 rows is slightly worse than the base model's rather than better.

Hence the dilution and the flatness are separate facts and neither rescues the mechanism. The batching does dilute the context severely, and sharpening it to a single borrower makes attention more uniform rather than less.

3.11.3 The context is a restatement of the base model's own prediction

The third explanation is the one that ties the other two together. Retrieval runs in the CLS token space, and the CLS token is what the decoder reads, so borrowers retrieved as neighbours are precisely those the base model already scores alike. If the mean response of a target's neighbours tracks the base model's own prediction, the context is telling the model what it already says.

▶ What the retrieved context knows

```
per 200-target batch, 149 batches
  retrieved-context default rate: mean 0.2874, sd 0.0330, range 0.2080 to
0.4000
  learning-sample default rate:   0.2888
  correlation with the base model's own predicted PD: +0.6535
```

Retrieval does find neighbourhoods with genuinely different default rates, from 0.208 to 0.400 across target batches against a learning-sample rate of 0.2888, so it is not simply returning the portfolio average. However that variation correlates at $+0.65$ with the base model's own predicted probability for the same targets, which is what searching the decoder's own space should produce.

Put the three together and the mechanism's problem is fully visible. The decorator injects each neighbour's outcome with a weight section 3.2 showed to be constant. Attention then averages those injections almost uniformly, whether over a diluted thousand-row sample or over a borrower's own sixty-four. And the quantity that

survives the averaging is a neighbourhood mean two-thirds explained by what the base model already predicts. So the layer is offered a noisy copy of its own output and correctly learns to ignore it.

This also suggests where the architecture would pay, and it is a testable prediction rather than a hedge. In-context learning has something to add when the retrieved neighbours carry information the base model cannot use: an unseen category, for instance, which section 4 turns to next.

4 Cold start: a country with no history

4.1 Bondora's Slovak book

The course's zero-shot experiment remaps French regions totalling 10 per cent of exposure to an *unseen* category and asks whether in-context learning can price them. Credit supplies the same experiment without the remapping, because a lender's segments genuinely appear and disappear.

▶ The four country books

Country	loans	default_rate	first	last
EE	86224	0.1703	2009-02-28	2020-07-19
FI	35879	0.3784	2013-07-23	2020-03-24
ES	26268	0.5543	2013-10-18	2020-03-24
SK	296	0.7061	2014-03-13	2015-01-05

Slovakia is a closed cohort. Originations run for ten months and then stop, which is what a withdrawal from a market looks like in a loan book, and the 296 loans that remain default at 70.6 per cent against Estonia's 17.0. Consequently it is a hard cold start rather than a cosmetic one: a model trained on the other three countries has to price a segment whose base rate is four times the largest book's.

4.2 The unseen-country setup

Slovakia is relabelled *unseen* and held out entirely, so every Slovak loan is a test row and none is available for fitting. A random 2 per cent of the three trained countries is relabelled *unseen* as well, which is what gives the level any training signal at all and mirrors what the course does with its small-exposure regions.

▶ Holding Slovakia out

```
learn 118,702 loans, 2,929 of them relabelled unseen
test  296 Slovak loans, default rate 0.7061, against a learning-sample
rate of 0.2879
```

4.3 Four models, with intervals

A point deviance on 296 rows is not readable, so every figure below carries a bootstrap interval over the test rows. Reporting one without the other would invite a comparison the sample cannot support.

▶ Null, GLM, base CT and ICL-CT on the held-out Slovak book

```
held-out Slovak book, 296 loans, observed default rate 0.7061
       model  deviance        lo        hi  mean_pred       AUC
  null model  195.7808  185.9921  204.9577     0.2879       NaN
logistic GLM  185.7197  175.5231  195.4379     0.3084    0.6233
     base CT  236.6256  219.5679  253.3656     0.2189    0.5497
      ICL-CT  234.6357  217.7763  251.2011     0.2240    0.5529

phase two best epoch 6, phase three best epoch 0
```

Separate intervals on two models scored over the same 296 loans understate how precisely their difference is known, because the resample is common to both. So the ICL-CT against the base model gets a paired bootstrap of its own.

▶ Paired bootstrap of the in-context gain

```
ICL-CT minus base CT   -1.990   95% CI [-2.913, -1.052]   improves in
100.0% of resamples
ICL-CT minus null      +38.855   95% CI [+28.930, +49.170]
```

Four things happen on the Slovak book and three of them are failures.

Every model under-predicts, badly. The observed default rate is 70.6 per cent and the mean predicted probabilities run from 21.9 to 30.8 per cent. None of the four models comes close, which is what a cold start looks like when the new segment's risk is genuinely different rather than merely unobserved. No amount of in-context learning fixes a level error of forty percentage points.

Both neural models are decisively worse than the null model. The base Credibility Transformer scores 236.6 and the ICL-CT 234.6, against the null model's 195.8, and neither interval comes near the null's. Predicting the portfolio average for every Slovak applicant would have been substantially better than either fitted network. That is the single most useful number in this lecture for anyone about to score a new segment with a model trained elsewhere.

Only the GLM beats the null, and only just. Its 185.7 against 195.8 is an improvement of about ten deviance points, and the two intervals overlap, so even that edge is not established on 296 loans. Its discrimination is real if modest at an AUC of 0.623, where both networks sit near 0.55 and are therefore close to uninformative on this segment. The reason is worth naming: the GLM's country effect enters as one coefficient on an *unseen* level estimated from the 2,929 relabelled training rows, whereas the networks learn an embedding for that level and then interact it with everything else, which is exactly the flexibility that has nothing to generalise from here.

In-context learning helps here, by about two deviance points, and the sign depends on the training budget. The paired difference is -1.99 with a bootstrap interval that excludes zero and every one of 2,000 resamples improving, so on the sampling uncertainty of these 296 loans the gain looks settled. It is not. Training the same mechanism to a longer budget reverses it.

▶ The same mechanism, trained to a longer budget

```
phase-two budget 20 epochs x 50 steps: ICL-CT 234.636, difference -1.990,
best epoch 6
phase-two budget 30 epochs x 100 steps: ICL-CT 240.798, difference
+4.172, best epoch 11
  the longer budget's paired 95% CI [+3.450, +4.920], improves in 0.0% of
resamples
```

The shorter budget stops at epoch 6 and gains two points; the longer one stops at epoch 11 and loses four, with a bootstrap interval just as tight and not one resample in two thousand pointing the other way. Both intervals are honest about what they measure, which is sampling variation in a 296-row test set, and neither measures the uncertainty that actually dominates here, namely which epoch model selection lands on.

Hence the lesson for a validation function is about the interval rather than the architecture. A paired bootstrap on a small out-of-time or out-of-segment sample can report a confident difference that reverses under a change to the training schedule, so a confidence interval on the test sample is not evidence that a model comparison is settled. Refit under more than one schedule before quoting either.

4.4 Where the context comes from

An in-context prediction is built out of retrieved neighbours, so for a Slovak applicant it is worth asking whose experience is actually being used. The pool contains no Slovak loans by construction, so the answer has to be somebody else's.

▶ The country composition of the retrieved context

```
retrieved context by country, pooled over every target batch
EE    0.710
ES    0.149
FI    0.141

learning-sample composition for comparison
EE    0.5810
FI    0.2415
ES    0.1776
```

Estonia supplies 71.0 per cent of the retrieved context against 58.1 per cent of the pool it is drawn from, so retrieval over-weights it by about thirteen percentage points, while Finland is under-weighted by ten. Estonia is the pool's most benign book at a 17.0 per cent default rate, and Slovakia, the segment being priced, is its most adverse at 70.6 per cent.

Hence the mechanism imports the wrong country's experience, and it does so for a structural reason rather than by accident. Retrieval measures similarity in the CLS token space, which encodes what the base model has learned to predict, and the base model predicts Slovak applicants to be low-risk because it has never seen a Slovak default. Consequently its nearest neighbours are the applicants it also scores low, which are disproportionately Estonian, and the retrieved outcomes then confirm the prediction that selected them. So a retrieval mechanism whose similarity metric comes from the model being corrected cannot correct that model where it is wrong about a whole segment.

That is a design finding rather than a bug in this implementation, and it suggests the fix: retrieve in a space that does not depend on the fitted model, for instance on raw standardised covariates or on a within-segment metric. This lecture does not build that variant, and lecture `R3`'s point about sample design is the more immediate lesson, namely that a segment whose base rate differs by a factor of four is not represented by the pool at all, whatever metric is used to search it.

5 What in-context learning costs in governance

Sections 2 to 4 measured what these models deliver. This section measures what they cost, and the costs are specific to credit rather than general to machine learning, because a lending decision has to be explainable to the person it is made about and a rating system has to be documented to a supervisor.

5.1 The explanation is other borrowers

Lecture 9 built a per-decision decomposition on this book and tested it. Its finding was quantitative: among the highest-PD decile the leading adverse characteristic was identical across all ten refits for 92.0 per cent of applicants, while the second-ranked one agreed for 1.6 per cent, because the leading contribution exceeded the second by 1.043 on the log-odds scale and the second exceeded the third by 0.118. One reason code is therefore defensible on this book and three are not.

An in-context prediction complicates that, because it is a function of the retrieved neighbours as well as of the applicant's own characteristics. The worry is therefore that a full account of what moved the score would have to name which other borrowers were in the pool, which is not something a lender can put in an adverse-action notice.

That is an argument from the construction, and this series does not stop at arguments from construction. The measurable version asks how much a borrower's prediction depends on which other borrowers happened to be in the pool, and the test is to re-draw it.

5.2 Reason-code stability under a context re-draw

The retrieval pool is resampled ten times, taking a random half of the learning sample on each draw, and every test borrower is rescored. Lecture 9 supplies the yardstick: on this book the leading adverse contribution exceeded the second by 1.043 on the log-odds scale and the second exceeded the third by 0.118. So a borrower whose log-odds moves by less than 0.118 keeps its reason-code ordering intact, and one that moves by more than 1.043 has no stable leading reason at all.

The model measured has to be one that is actually doing in-context learning, so section 3.11's actively trained state is the one re-scored below. Section 3.9's selected state is the identity initialisation, whose ICL layers return the target token untouched, so its prediction does not depend on the pool at all and re-drawing it moves nothing by construction. Both are reported, the trained state as the measurement and the selected state as a check that the measurement is wired up correctly.

▶ Ten pool re-draws on 2,000 test borrowers

```
section 3.11's actively trained state: 10 re-draws, 2,000 borrowers
  per-borrower log-odds sd     mean 0.0007   median 0.0006   90th pct
0.0011
  per-borrower log-odds range  mean 0.0023   90th pct 0.0036   max 0.0072
  share moving more than 0.118 (lecture 9's second-to-third gap): 0.0%
  share moving more than 1.043 (its leading-to-second gap):       0.0%

section 3.9's selected state, which ignores its context: 10 re-draws,
2,000 borrowers
  per-borrower log-odds sd     mean 0.0000   median 0.0000   90th pct
0.0000
  per-borrower log-odds range  mean 0.0000   90th pct 0.0000   max 0.0000
  share moving more than 0.118 (lecture 9's second-to-third gap): 0.0%
  share moving more than 1.043 (its leading-to-second gap):       0.0%
```

The concern does not materialise on this book, and the reason it does not is the same fact that produced section 3's null result.

Re-drawing half the retrieval pool moves a borrower's fitted log-odds by about 0.002 in range and 0.0007 in standard deviation. Set against lecture 9's 0.118 gap between the second and third reason codes, that is a fiftieth of the smallest gap the reason-code ordering has to survive, and no borrower in the 2,000 examined moves by even that much. The selected state moves by exactly zero, which confirms the measurement is reading the mechanism rather than numerical noise.

The mechanism is stable for the two reasons section 3.11 measured. Its attention row is nearly uniform, so the quantity reaching the decoder is close to the mean outcome of the retrieved set, and a mean over a thousand rows barely moves when half the pool is removed. Furthermore the retrieved set was never a borrower's own neighbourhood in the first place, since the frequency cap leaves the median borrower three of its sixty-four neighbours, so resampling the pool exchanges one near-random thousand-row sample for another and the two have almost the same mean. Consequently the same properties that stopped in-context learning from improving the deviance also stop it from destabilising the explanation. A version of this architecture that attended sharply to a handful of neighbours would buy more accuracy and would put the reason code back at risk, so the two properties trade against each other and a lender cannot have the first without testing for the second.

THE GENERAL LESSON, WHICH IS ABOUT THE TEST RATHER THAN THE ANSWER

The re-draw is cheap, it takes ten forward passes, and it converts an argument about explainability into a number on the same scale as the reason codes it threatens. Run it on any retrieval-based credit model before deciding whether the model can support an adverse-action notice, because the answer is a property of that model's attention rather than of the architecture in general, and this lecture's answer is reassuring only because its mechanism is doing so little.

5.3 What does bite: the pool's composition

Sampling noise inside the pool turns out to be harmless. The pool's **composition** is not, and section 4.5 measured that: retrieval drew 71.0 per cent of its context from Estonia against 58.1 per cent of the available pool, and it did so when pricing the segment whose default rate is four times Estonia's.

So the governance question that survives is not "does this prediction wobble when the pool is resampled?" but "whose experience is in the pool, and is it the right population for this applicant?" That is a representativeness question, and it is the one Article 174(c) of the Capital Requirements Regulation (CRR) already asks.

5.4 A prior whose training data cannot be shown

Section 2's TabPFN raises a different problem, and it is the sharper one for a regulated rating system. The model's weights were fitted on tasks drawn from a synthetic prior, so there is no training population to describe.

Lecture `R3` established the anchor. **CRR Article 174(c)** requires that the population of exposures represented in the data used for estimation, the lending standards used when the data was generated, and other relevant characteristics be comparable with those of the firm's actual exposures and standards. A lender using a tabular foundation model can satisfy that requirement for its **context** dataset, which is its own book. It cannot satisfy it for the pretrained weights, because the comparison Article 174(c) demands is

between two populations and one of them is a prior over data-generating mechanisms rather than a population of exposures.

Two readings follow and this lecture does not choose between them, because the choice is a supervisory judgement rather than a statistical one. On the first, the pretrained weights are part of the estimation method rather than part of the data, in the way that the choice of link function is, and Article 174(c) then bites on the context set alone. On the second they are data by another name, since they are what makes the prediction, and the requirement cannot be met at all. The bureau-score precedent of section 1.3 favours the first reading, and a bureau does at least publish a development population.

> **WHAT IS SETTLED**
>
> Whichever reading a supervisor takes, three things follow from section 2's measurements rather than from any interpretation. The model arrives about 1.9 percentage points below the observed portfolio default rate, so it needs recalibration before it prices anything. It has no fitted parameter vector to document or monitor, so the usual apparatus of parameter stability testing has nothing to test. And its advantage over a conventional model disappeared by 10,000 observations, so on a book of any size the question of whether it may be used is mostly academic.

5.5 Data protection

One further consequence deserves stating, because it is easy to miss inside an architecture diagram. The context batch carries **other borrowers' observed outcomes** into this applicant's decision. In the fixed-horizon setting those outcomes are defaults, which is personal data about identified individuals, and they enter the decision at prediction time rather than having been absorbed into a parameter during fitting.

UK GDPR **Articles 22A to 22D**, substituted for the former Article 22 by the Data (Use and Access) Act 2025 and in force in that form since 5 February 2026, govern significant decisions based solely on automated processing. Lecture 9 records the verification, and it is worth repeating the warning attached to it: much of the secondary literature still

quotes the superseded Article 22 wording. The safeguards those articles require include information about the decision and the right to contest it, and section 5.2's measurement is directly relevant to whether those safeguards can be met, since a decision that survives a re-draw of the pool can be explained by reference to the applicant.

Two points survive the measurement all the same, and both are structural. The retrieved outcomes are still other people's defaults, used at prediction time rather than absorbed into a parameter during fitting, so the processing happens later in the pipeline than a lender's data inventory usually assumes. And a lender cannot rely on section 5.2's answer without measuring it, because the answer came out reassuring only because the attention was flat. This lecture therefore states what it measured rather than a general clearance for the architecture.

Takeaways and outlook

Two ways of borrowing experience were tested against one question, and they answered it differently.

A pretrained prior pays where a lender has no history. TabPFN v2 led the learning curve at every sample size up to 10,000 rows, and at 250 rows it closed 8.8 deviance points against the null model where a network trained on the same data closed 2.5. The classical estimator did not merely trail there; it separated, averaging a deviance of 1,901 at 250 rows and behaving erratically up to 2,000, because maximum likelihood on thirty parameters and a few hundred observations has no mechanism to shrink. Consequently the case for a cross-table prior on a small book is a practical one rather than a fashionable one.

What the prior does not do is beat a model built on data. TabPFN handed 10,000 rows reached 110.88 and a logistic GLM fitted on all 118,933 reached 110.87, so eight per cent of the data with no fitting at all reproduced the model a lender would build on the whole of it, and the Credibility Transformer on the whole of it took another half point. Furthermore the foundation model arrived 1.9 percentage points below the observed

default rate, about 2.3 sampling standard errors, and it has no parameter through which that can be corrected, so lecture 7's recalibration has to sit outside the model as a mapping the lender owns.

In-context learning did nothing on the application book, and the reason is measurable rather than mysterious. Five paired fits put the ICL-CT 0.012 behind its own base model with a paired standard deviation of 0.032, and in two of the five seeds no epoch of either training phase beat the identity initialisation.

Three measurements explain that, and the first was a surprise. The paper's batch retrieval unions 200 targets' 64 nearest neighbours and caps the union at 1,000, and on this book those neighbourhoods hardly overlap: 95 per cent of retrieved rows are retrieved by exactly one target, so the cap discards 92 per cent of the union and hands the median borrower **three of its own sixty-four neighbours**. The context is therefore close to a random thousand-row sample of the book. Sharpening it does not help either, since scoring one borrower at a time against its own 64 neighbours makes the attention row flatter still, a coefficient of variation of 0.02 against 0.10, and moves the prediction to a slightly worse place rather than a better one. And what survives the averaging is a neighbourhood mean correlating 0.65 with what the base model already predicts, because retrieval searches the very space the decoder reads. The layer is offered a noisy copy of its own output and correctly learns to ignore it.

The transfer also loses a component on the way in. The decorator's credibility weight is $v/(v+\kappa)$ in exposure, and a fixed-horizon table gives every loan the same twelve-month window, so the weight collapses to a constant and κ becomes a shrinkage scalar. A lender wanting the credibility reading back has to move to the survival table, which changes the estimand and belongs to the `S` track.

The cold start is where the series' models fail hardest. On 296 Slovak loans defaulting at 70.6 per cent, both networks scored worse than predicting the portfolio average, 236.6 and 234.6 against 195.8, and only the GLM beat it and only within overlapping intervals. Every model under-predicted by around forty percentage points. Retrieval made the problem worse in a way worth understanding: it drew 71.0 per cent of its context from Estonia against 58.1 per cent of the available pool, over-weighting the most benign book while pricing the most adverse segment, because similarity was measured in the space of the model that was wrong about the segment. A retrieval

metric taken from the fitted model cannot correct that model where it is wrong about a whole population.

One methodological finding came out of the same experiment and travels further than any of the above. The paired bootstrap reported a confident two-point gain under one training budget and a confident four-point loss under a longer one, with neither interval containing zero and no resample dissenting in either direction. So a confidence interval computed on a small test sample measures sampling variation and not model-selection variation, and on a 296-row segment the second dominates. Refit under more than one schedule before quoting either number.

The governance bill turned out smaller than the architecture suggests. Re-drawing half the retrieval pool moved a borrower's fitted log-odds by 0.002, against the 1.043 that separates lecture 9's leading reason code from its second, and no borrower among 2,000 moved even as far as the 0.118 that separates the second from the third. So the reason code survives. It survives because the attention is flat, however, which is the same property that denied the mechanism any accuracy, and a sharper variant would put the explanation back at risk. Hence the re-draw is ten forward passes and belongs in the validation pack of any retrieval-based credit model, rather than being assumed either way.

The unresolved question is TabPFN's, and it is not statistical. A model pretrained on synthetic tasks has no training population to compare with the lender's exposures, which is what CRR Article 174(c) asks for, and whether the pretrained weights count as method or as data is a supervisory judgement this lecture declines to make on a supervisor's behalf.

Three threads run on from here. Lecture `C2` still owes an account of when an attribution may be read causally, and section 5.2's re-draw is a stability test rather than a causal one, so it belongs on that list. Lecture `R3`'s representativeness machinery is what section 4 kept running into, and a version of that lecture written after this one would treat a retrieval pool as a sample to be designed rather than a database to be searched. And the retrieval metric itself is the obvious experiment this lecture did not run: searching on standardised covariates rather than on CLS tokens would test whether section 4.5's Estonian bias is a property of in-context learning or only of retrieving in the fitted model's own space.

Copyright and attribution

The architecture, the mathematics, and the notation are those of Scognamiglio, Maggi, Wüthrich and Richman, from lecture 12 of the summer school *Deep Learning for Actuarial Modeling* and from the lecture notes at SSRN 5162304, provided under CC BY-NC. The credit adaptation, the implementation, the datasets, and every measurement reported above are Mario Ervedosa's, and any error in them is his.

References

- Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33.
- Bühlmann, H. (1967). Experience rating and credibility. *ASTIN Bulletin*, 4(3), 199-207.
- Gorishniy, Y., Rubachev, I., Khrulkov, V. and Babenko, A. (2021). Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems*, 34. *arXiv:2106.11959*.
- Hoffmann, J., Borgeaud, S., Mensch, A. et al. (2022). Training compute-optimal large language models. *arXiv:2203.15556*.
- Hollmann, N., Müller, S., Eggensperger, K. and Hutter, F. (2022). TabPFN: a transformer that solves small tabular classification problems in a second. *arXiv:2207.01848*.
- Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., Körfer, M., Hoo, S.B. et al. (2025). Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045), 319-326.
- Huang, X., Khetan, A., Cvitkovic, M. and Karnin, Z. (2020). TabTransformer: tabular data modeling using contextual embeddings. *arXiv:2012.06678*.

- Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B. et al. (2020). Scaling laws for neural language models. *arXiv:2001.08361*.
- Müller, S., Hollmann, N., Pineda Arango, S., Grabocka, J. and Hutter, F. (2021). Transformers can do Bayesian inference. *arXiv:2112.10510*.
- OpenAI (2023). GPT-4 technical report. *arXiv:2303.08774*.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P. et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35.
- Padayachy, K., Richman, R., Scognamiglio, S. and Wüthrich, M.V. (2025). In-context learning enhanced credibility transformer. *arXiv:2509.08122*.
- Qu, J., Holzmüller, D., Varoquaux, G. and Le Morvan, M. (2025). TabICL: a tabular foundation model for in-context learning on large data. *International Conference on Machine Learning*. *arXiv:2502.05564*.
- Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I. (2018). Improving language understanding by generative pre-training. *OpenAI technical report*.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI technical report*.
- Richman, R., Scognamiglio, S. and Wüthrich, M.V. (2025). The credibility transformer. *European Actuarial Journal*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Wang, Z. and Sun, J. (2022). TransTab: learning transferable tabular transformers across tables. *arXiv:2205.09328*.
- Wüthrich, M.V., Merz, M., Richman, R., Scognamiglio, S. and Maggi, M. (2025). AI Tools for Actuaries. *SSRN Manuscript 5162304*.